# Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

**Thème principal :** Algorithmique et Programmation - Structures Itératives

**Synthèse exécutive :** Ce cours aborde les structures répétitives (boucles) en algorithmique et en langage C. L'objectif est de comprendre pourquoi et comment automatiser la répétition d'instructions sans duplication de code, en respectant le principe DRY (Don't Repeat Yourself). Le cours détaille le fonctionnement, les cas d'usage stricts et la syntaxe des trois boucles fondamentales : 'while' (tant que), 'do-while' (faire... tant que) et 'for' (pour). Un accent particulier est mis sur la prévention des boucles infinies, la gestion des conditions d'arrêt (pré-condition vs post-condition) et l'alignement avec les bonnes pratiques d'évaluation en BUT Informatique.

## 1. Utilité des structures itératives et principe DRY

En programmation, répéter un bloc d'instructions ne doit jamais être réalisé par du copier-coller. Dupliquer 10 000 fois une ligne de code rend le programme illisible, lourd, impossible à maintenir et extrêmement sujet aux erreurs. Les structures itératives (ou boucles) permettent d'exécuter un même ensemble d'instructions de manière répétée tant qu'une condition logique (expression booléenne) reste vraie. L'enjeu central lors de la conception d'une boucle est la maîtrise de la condition d'arrêt et la mise à jour effective de la variable de contrôle. Si la variable de contrôle n'est pas modifiée au sein du corps de la boucle, la condition d'arrêt reste indéfiniment vraie, entraînant une boucle infinie qui gèle le programme et consomme inutilement les ressources du processeur.

- Principe DRY (Don't Repeat Yourself) : interdiction fondamentale de dupliquer du code par copier-coller.
- Condition d'arrêt : expression booléenne déterminant la poursuite ou la sortie de la boucle.
- Variable de contrôle / Compteur : variable impérativement mise à jour au fil des itérations.
- Boucle infinie : blocage du programme causé par une condition d'arrêt qui ne devient jamais fausse.

***Cas concret enseignant :*** *Métaphore du tapis roulant d'usine d'emballage : tant qu'un colis est détecté par le capteur (condition vraie), le bras robotisé l'attrape et l'emballe (instruction). Si le capteur ne détecte plus de colis ou si le compteur atteint le quota fixé, le robot s'arrête. Sans condition de fin fonctionnelle, le robot continuerait à tourner à vide et finirait par surchauffer et détruire la machine.*

## 2. La boucle TANT QUE (while) : Structure à pré-condition

La structure 'while' est la boucle de choix lorsque le nombre précis d'itérations n'est pas connu à l'avance et dépend d'un événement dynamique (saisie utilisateur, fin de lecture d'un fichier, signal réseau). La condition étant testée dès l'entrée, si celle-ci s'avère fausse lors du premier contrôle, l'intérieur de la boucle n'est jamais exécuté (0 itération). Pour garantir l'arrêt de la boucle, il est absolument obligatoire que le corps de la boucle modifie la valeur d'au moins une variable intervenant dans la condition d'évaluation.

- Pré-conditionnement : la condition d'accès est évaluée AVANT chaque passage dans le bloc d'instructions.
- Nombre d'itérations : de 0 à N fois (le bloc peut ne jamais s'exécuter si la condition est fausse d'entrée).
- Initialisation préalable : toute variable testée dans la condition doit posséder une valeur définie avant l'instruction while.

***Cas concret enseignant :*** *Exemple du contrôle du code PIN au distributeur de billets : la machine demande le code TANT QUE le code saisi est incorrect ET QUE le nombre d'essais est inférieur à 3. Si l'utilisateur saisit le bon code du premier coup, la boucle s'arrête immédiatement sans répéter la demande.*

## 3. La boucle FAIRE... TANT QUE (do-while) : Structure à post-condition

Contrairement à la boucle 'while', la structure 'do-while' exécute toujours son corps d'instructions une première fois avant d'évaluer la condition de maintien. Elle est idéalement adaptée au contrôle de saisie (pour forcer l'utilisateur à entrer une donnée valide dans un intervalle précis) et aux menus interactifs. En langage C, sa syntaxe impose un point-virgule final : do { ... } while (condition);. L'absence de ce point-virgule constitue l'une des erreurs de compilation les plus fréquentes commises par les étudiants.

- Post-conditionnement : le test logique est réalisé APRÈS l'exécution du bloc d'instructions.
- Garantie d'exécution : la boucle s'exécute obligatoirement au minimum 1 fois (1 à N itérations).
- Syntaxe C stricte : obligation impérative d'apposer un point-virgule ';' après la parenthèse fermante de la condition while.

***Cas concret enseignant :*** *Cas du contrôle de saisie d'une note d'examen : 'Il est techniquement impossible de vérifier si la saisie faite par un étudiant est valide avant qu'il n'ait tapé au moins une valeur sur le clavier. On doit exécuter la saisie au moins une fois, ce qui fait du do-while la structure parfaitement adaptée.'*

## 4. La boucle POUR (for) : Itération déterminée

La structure 'for' rationalise la mécanique itérative en centralisant l'initialisation, le test de passage et la mise à jour de la variable de boucle en une seule ligne. Elle est systématiquement recommandée lorsqu'on doit effectuer un nombre d'itérations fixe et connu à l'avance (ex: parcourir les éléments d'un tableau de taille N). En C, les trois parties séparées par des points-virgules dans l'en-tête sont techniquement optionnelles, mais les omettre nuit à la lisibilité du code et est sanctionné en examen.

- En-tête condensé : regroupe initialisation, condition de maintien et incrémentation sous la forme for(init; cond; inc).
- Nombre d'itérations déterminé : privilégié lorsque le nombre exact de répétitions est connu avant d'entrer dans la boucle.
- Portée (scope) : possibilité de déclarer la variable d'itération directement au sein de l'en-tête (ex: for(int i = 0; ...)).

***Cas concret enseignant :*** *Métaphore du tour de piste d'athlétisme : 'Vous savez avant de vous lancer que vous devez effectuer exactement 5 tours. Vous initialisez votre compteur à 1 (init), vous vérifiez que vous n'avez pas dépassé 5 (cond), et à chaque franchissement de la ligne d'arrivée, vous ajoutez 1 à votre compteur (inc).'*

## ■ Questions & Interventions des Étudiants en Classe

**Question étudiant :** Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
**Réponse du professeur :** Parce que la condition d'arrêt repose sur l'évolution de la variable 'i'. Si 'i' ne change jamais de valeur, la condition reste éternellement vraie, ce qui plonge le programme dans une boucle infinie qui bloque le processeur.

**Question étudiant :** Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
**Réponse du professeur :** Oui, toute boucle 'for' peut techniquement s'écrire sous forme de boucle 'while'. Cependant, le 'for' est beaucoup plus lisible et adapté lorsque le nombre d'itérations est connu à l'avance grâce à son en-tête centralisé.

## ■ Exercice & Démonstration d'Application

**Énoncé :** Écrire un programme complet en langage C qui calcule la somme des N premiers entiers positifs (ex: si N = 5, calculer 1 + 2 + 3 + 4 + 5 = 15). Le programme doit obligatoirement valider que N est un entier strictement positif (> 0) à l'aide d'une boucle 'do-while', puis calculer le cumul au moyen d'une boucle 'for'.

**Résolution pas à pas :**
Étape 1 : Déclarer les variables entières nécessaires : int n, somme = 0, i; Étape 2 : Mettre en place la saisie sécurisée du nombre N à l'aide d'une boucle do-while : do { printf("Entrez N (>0) : "); scanf("%d", &n;); } while (n <= 0); Étape 3 : Initialiser la variable de cumul somme = 0; Étape 4 : Mettre en place la boucle for pour cumuler les valeurs de 1 à N : for (i = 1; i <= n; i++) { somme += i; } Étape 5 : Afficher le résultat final avec printf("La somme est :

%d\n", somme);

**Résultat : Pour une saisie N = 5, la boucle 'do-while' valide la saisie positive dès le premier passage. La boucle 'for' effectue 5 itérations : à i=1 (somme=1), i=2 (somme=3), i=3 (somme=6), i=4 (somme=10), i=5 (somme=15). Le programme affiche 'La somme est : 15'. Si l'utilisateur saisit une valeur négative (ex: -3), la boucle do-while rejette la valeur et redemande la saisie.**

## ■ Auto-Évaluation Flash d'Examen

**Q : Quelle est la différence fondamentale d'évaluation entre une boucle while et une boucle do-while ?**

*R : La boucle while évalue sa condition avant l'exécution du bloc (0 à N itérations), tandis que la do-while l'évalue après, garantissant au moins 1 exécution (1 à N itérations).*

**Q : Que se passe-t-il si la condition initiale d'une boucle for est fausse dès la première évaluation ?**

*R : Le bloc d'instructions interne n'est exécuté aucune fois (0 itération), exactement comme pour un while dont la pré-condition est fausse d'entrée.*

**Q : Quel oubli de syntaxe systématique provoque une erreur de compilation sur la boucle do-while en langage C ?**

*R : L'oubli du point-virgule final juste après la parenthèse fermante de la condition while (ex: do { ... } while(cond);).*